

Rand Protocol: A Permissionless Privacy Preserving Transactional System

A Hybrid Proof-of-Useful-Work and Byzantine Fault Tolerant Consensus Protocol
with Privacy-Preserving Transactions on the Solana Virtual Machine

Dendi Suhubdy
Bitwyre Research
dendi@bitwyre.com

December 21, 2025

Abstract

We present Rand, a novel blockchain architecture that combines Proof-of-Useful-Work (PoUW) with Byzantine Fault Tolerant (BFT) consensus to achieve both permissionless participation and fast transaction finality. Our system introduces a dual-token economy where validators earn ATLAS tokens through BFT consensus participation, while GPU provers earn SHRUG tokens by generating zero-knowledge proofs for privacy-preserving transactions. The architecture leverages the Solana Virtual Machine (SVM) for smart contract execution, enabling interoperability with the broader Solana ecosystem. We demonstrate that our hybrid approach achieves 2-6 second deterministic finality while maintaining permissionless entry through GPU proving, effectively transforming the wasteful computation of traditional Proof-of-Work into useful cryptographic work. We provide formal security analysis under standard cryptographic assumptions and demonstrate through theoretical analysis that the system can achieve throughput exceeding 5,000 transactions per second while supporting post-quantum secure private transactions using STARK proofs. The protocol ensures full supply auditability through on-chain verification mechanisms and transparent emission schedules.

Keywords: Blockchain, Byzantine Fault Tolerance, Zero-Knowledge Proofs, Privacy, Proof-of-Useful-Work, Solana Virtual Machine, Supply Auditability

Contents

1	Introduction	4
1.1	Contributions	4
1.2	Paper Organization	4
2	Preliminaries	5
2.1	Notation	5
2.2	Cryptographic Hash Functions	5
2.3	Digital Signatures	5
2.4	Commitment Schemes	5
2.5	Zero-Knowledge Proofs	6
2.6	Byzantine Fault Tolerance	6

3	System Architecture	6
3.1	Overview	6
3.2	Dual-Token Model	6
3.3	Rationale for Dual Gas Tokens	7
3.4	Economic Separation of Concerns	7
3.5	Resource Accounting	7
3.6	Participant Roles	8
3.7	Solana Virtual Machine Integration	8
4	Consensus Protocol	8
4.1	HotStuff BFT	8
4.2	Validator Set Management	9
4.3	Leader Selection	9
4.4	Slashing Conditions	9
5	Privacy-Preserving Transactions	9
5.1	Private State Model	9
5.2	Private Transaction Circuit	10
5.3	STARK Proof System	10
5.4	Encrypted Outputs	10
6	Supply Auditability and Transparency	11
6.1	Motivation for Supply Auditability	11
6.2	ATLAS Supply Auditability	11
6.3	SHRUG Supply Auditability	11
6.4	Privacy-Preserving Supply Verification	12
7	Zero-Knowledge Proof Generation Pipeline	12
7.1	Overview	12
7.2	Step 1: Rust Circuit Definition	12
7.3	Step 2: Algebraic Intermediate Representation (AIR)	13
7.4	SVM Instruction ZK-Compatibility	13
7.5	Performance Characteristics	13
7.6	End-to-End Proving Timeline	13
8	Network Architecture	14
8.1	Maximum Block Size and Packet Specifications	14
8.2	Gossip Protocol Packet Structure	14
8.3	Performance Metrics	14
8.4	Bandwidth Requirements	14
9	Token Economics	15
9.1	Initial Token Distribution	15
9.2	Inflation Model	15
9.3	SHRUG Demand-Driven Emission	15
9.4	Fee Distribution	15
10	Security Analysis	15
10.1	Consensus Security	15
10.2	Privacy Security	16
10.3	Post-Quantum Security	16

11 Implementation Considerations	16
11.1 Development in Rust	16
11.2 Performance Targets	17
12 Related Work	17
13 Conclusion	17

1 Introduction

The blockchain trilemma—the challenge of simultaneously achieving decentralization, security, and scalability—has driven extensive research into consensus mechanism design [5]. Traditional Proof-of-Work (PoW) systems like Bitcoin [14] provide strong security guarantees and permissionless participation but suffer from limited throughput and probabilistic finality. Proof-of-Stake (PoS) variants [10, 7] improve efficiency but often introduce barriers to entry through minimum stake requirements.

Privacy in blockchain transactions presents an orthogonal challenge. While systems like Zcash [9] and Monero [15] provide transaction privacy, they typically sacrifice smart contract functionality or rely on trusted setups. Recent advances in zero-knowledge proof systems, particularly STARKs [1], enable transparent, post-quantum secure proofs suitable for privacy-preserving computation.

We introduce Rand, a blockchain architecture that addresses these challenges through:

1. **Hybrid PoUW-BFT Consensus:** Combining permissionless GPU proving with deterministic BFT finality
2. **Dual-Token Economy:** Separate incentive mechanisms for validators (ATLAS) and provers (SHRUG)
3. **Privacy-Preserving Transactions:** STARK-based private transfers with post-quantum security
4. **SVM Compatibility:** Full interoperability with Solana smart contracts and tooling

1.1 Contributions

Our main contributions are:

- A formal specification of the PoUW-BFT hybrid consensus protocol achieving $O(n)$ message complexity
- Security proofs demonstrating Byzantine fault tolerance with up to $f < n/3$ faulty validators
- A privacy-preserving transaction model using STARK proofs with formal privacy guarantees
- Economic analysis of the dual-token incentive mechanism
- Integration architecture for the Solana Virtual Machine
- Complete supply auditability framework for both ATLAS and SHRUG tokens
- Comprehensive ZK proof generation pipeline from Rust to STARK proofs
- Detailed network architecture with performance specifications

1.2 Paper Organization

Section 2 provides cryptographic preliminaries. Section 3 describes the system architecture. Section 4 presents the consensus protocol. Section 5 details the privacy-preserving transaction mechanism. Section 6 explains supply auditability. Section 7 covers the ZK proof generation pipeline. Section 8 describes network architecture. Section 9 analyzes token economics. Section 10 provides security analysis. Section 11 discusses implementation. Section 12 reviews related work, and Section 13 concludes.

2 Preliminaries

2.1 Notation

Let λ denote the security parameter. We write $x \leftarrow S$ for uniform sampling from set S and $x \leftarrow \mathcal{A}(\cdot)$ for the output of algorithm $\mathcal{A}$. We use $\{0,1\}^*$ for arbitrary-length bit strings and $\{0,1\}^\lambda$ for λ -bit strings. $\epsilon(\lambda)$ denotes a negligible function.

2.2 Cryptographic Hash Functions

Definition 2.1 (Collision-Resistant Hash Function). *A hash function family $\mathcal{H} = \{H_\lambda : \{0,1\}^* \rightarrow \{0,1\}^\lambda\}_{\lambda \in \mathbb{N}}$ is collision-resistant if for all probabilistic polynomial-time (PPT) adversaries $\mathcal{A}$:*

$$\Pr[(x, x') \leftarrow \mathcal{A}(1^\lambda) : x \neq x' \wedge H_\lambda(x) = H_\lambda(x')] \leq \epsilon(\lambda) \quad (1)$$

We employ SHA-3 (Keccak-256) [2] and BLAKE3 [16] as our primary hash functions, both providing 128-bit post-quantum security against Grover’s algorithm [8].

2.3 Digital Signatures

Definition 2.2 (Digital Signature Scheme). *A digital signature scheme $\Sigma = (\text{KeyGen}, \text{Sign}, \text{Verify})$ consists of:*

- $\text{KeyGen}(1^\lambda) \rightarrow (pk, sk)$: *Key generation*
- $\text{Sign}(sk, m) \rightarrow \sigma$: *Signing algorithm*
- $\text{Verify}(pk, m, \sigma) \rightarrow \{0,1\}$: *Verification algorithm*

satisfying correctness and existential unforgeability under chosen message attacks (EUF-CMA).

For post-quantum security, we employ CRYSTALS-Dilithium [6], a lattice-based signature scheme standardized by NIST. Dilithium signatures provide:

- Public key size: 1,312 bytes
- Signature size: 2,420 bytes
- Security level: NIST Level 2 (128-bit quantum security)

2.4 Commitment Schemes

Definition 2.3 (Commitment Scheme). *A commitment scheme $\text{Comm} = (\text{Setup}, \text{Commit}, \text{Open})$ satisfies:*

- **Hiding:** *For all PPT adversaries, $\text{Comm}(m_0; r_0)$ is computationally indistinguishable from $\text{Comm}(m_1; r_1)$*
- **Binding:** *No PPT adversary can find (m, r) and (m', r') with $m \neq m'$ such that $\text{Comm}(m; r) = \text{Comm}(m'; r')$*

We use Pedersen commitments over elliptic curves for efficiency:

$$\text{Comm}(v; r) = v \cdot G + r \cdot H \quad (2)$$

where $G, H \in \mathbb{G}$ are independent generators of a group of prime order q .

2.5 Zero-Knowledge Proofs

Definition 2.4 (Zero-Knowledge Argument System). *A zero-knowledge argument system for relation R consists of (Prove, Verify) satisfying:*

- **Completeness:** For $(x, w) \in R$, $\Pr[\text{Verify}(x, \text{Prove}(x, w)) = 1] = 1$
- **Soundness:** For $x \notin L_R$ and PPT $\mathcal{A}$: $\Pr[\text{Verify}(x, \mathcal{A}(x)) = 1] \leq \epsilon(\lambda)$
- **Zero-Knowledge:** $\exists$ PPT simulator $\mathcal{S}$ such that $\{\text{Prove}(x, w)\}_{(x, w) \in R} \approx_c \{\mathcal{S}(x)\}_{x \in L_R}$

We employ STARKs (Scalable Transparent Arguments of Knowledge) [1] for zero-knowledge proofs, providing:

- Transparent setup (no trusted ceremony)
- Post-quantum security (hash-based)
- Proof size: $O(\log^2 n)$ for computation of size n
- Verification time: $O(\log^2 n)$

2.6 Byzantine Fault Tolerance

Definition 2.5 (Byzantine Fault Tolerance). *A distributed protocol is Byzantine fault tolerant if it maintains safety and liveness properties in the presence of up to f Byzantine (arbitrarily malicious) nodes out of n total nodes, where $f < n/3$.*

The classical result of Lamport et al. [12] establishes that $n \geq 3f + 1$ nodes are necessary and sufficient for Byzantine agreement.

3 System Architecture

3.1 Overview

The Rand architecture consists of four primary layers:

1. **Consensus Layer:** HotStuff BFT + Validator Set
2. **Execution Layer:** Solana Virtual Machine (SVM)
3. **Privacy Layer:** STARK Proofs + Commitment Trees
4. **Network Layer:** P2P Gossip + Prover Pool

3.2 Dual-Token Model

The system operates with two native tokens:

Definition 3.1 (ATLAS Token). *The public utility token ATLAS serves as:*

- Gas payment for public transactions
- Staking collateral for validators
- Governance voting power
- Component of USDB stablecoin backing

Definition 3.2 (SHRUG Token). *The private utility token SHRUG serves as:*

- Gas payment for private transactions
- Reward for GPU provers generating ZK proofs
- Component of USDB stablecoin backing

3.3 Rationale for Dual Gas Tokens

A critical design decision in Rand is the separation of gas payment into two distinct tokens: ATLAS for public transactions and SHRUG for private transactions. This separation addresses several fundamental challenges in privacy-preserving blockchain design.

Theorem 3.3 (Gas Payment Privacy Leakage). *In single-token systems where private transactions pay gas in the same token as public transactions, an adversary can correlate private transaction senders with public identities through gas payment patterns.*

Proof. Consider a user with public identity id_{pub} holding tokens at address A . To execute a private transaction tx_{priv} , they must pay gas from A , creating the linkage:

$$\text{tx}_{\text{priv}} \xleftarrow{\text{gas payment}} A \xleftarrow{\text{ownership}} \text{id}_{\text{pub}} \quad (3)$$

This reveals that id_{pub} initiated tx_{priv} , defeating privacy. $\square$

Proposition 3.4 (Privacy-Preserving Gas). *SHRUG tokens can be held in shielded form and spent for gas within the zero-knowledge circuit, preserving sender anonymity:*

$$R_{\text{private_gas}} = \left\{ \begin{array}{l} \text{Valid private transfer proof} \\ \wedge \text{ Gas note nullifier valid} \\ \wedge \text{ Gas amount} \geq \text{required fee} \end{array} \right\} \quad (4)$$

3.4 Economic Separation of Concerns

The dual-token model creates distinct economic incentive structures:

1. **Independent Price Discovery:** ATLAS and SHRUG prices reflect their respective ecosystem demands
2. **Diversified Stablecoin Backing:** Low correlation between ATLAS and SHRUG prices provides diversification for USDB backing
3. **Specialized Incentives:** Validators optimize for consensus performance (ATLAS rewards), while GPU provers optimize for proof generation (SHRUG rewards)

3.5 Resource Accounting

Private transactions consume fundamentally different resources than public transactions:

Table 1: Resource Consumption by Transaction Type		
Resource	Public Tx	Private Tx
SVM computation	High	Low
State storage	Variable	Fixed (commitments)
Bandwidth	Low	High (proofs 50-200KB)
GPU proving	None	High (3-10 seconds)
Verification	Signature only	STARK verification

Separate gas tokens allow accurate pricing of these distinct resource profiles without cross-subsidization.

3.6 Participant Roles

1. **Validators:** Stake ATLAS tokens, participate in BFT consensus, earn ATLAS block rewards
2. **GPU Provers:** Generate STARK proofs for private transactions, earn SHRUG tokens
3. **Users:** Submit public or private transactions, pay fees in respective tokens
4. **Delegators:** Delegate ATLAS to validators, earn proportional rewards

3.7 Solana Virtual Machine Integration

We adopt the Solana Virtual Machine (SVM) [18] as our execution environment.

Table 2: EVM vs SVM Comparison

Feature	EVM	SVM
Execution model	Sequential	Parallel
State model	Unified account	Separated accounts
Typical TPS	15-30 (L1)	5,000-65,000
Smart contract language	Solidity, Vyper	Rust, C, C++
Deterministic fees	No	Yes

Definition 3.5 (SVM Transaction Parallelism). *Two transactions tx_1 and tx_2 can execute in parallel if and only if:*

$$(tx_1.write_accounts \cap tx_2.accounts = \emptyset) \wedge (tx_2.write_accounts \cap tx_1.accounts = \emptyset) \quad (5)$$

4 Consensus Protocol

4.1 HotStuff BFT

We employ HotStuff [19] as our BFT consensus protocol due to its linear message complexity and optimistic responsiveness. The protocol proceeds in views, each with a designated leader.

Definition 4.1 (Quorum Certificate). *A Quorum Certificate (QC) for block B at view v is a set of signatures from at least $2f + 1$ validators on the tuple $(B.hash, v, type)$ where $type \in \{prepare, precommit, commit\}$.*

Algorithm 1 HotStuff Consensus Protocol (Leader)

```
1: procedure LEADERPROTOCOL( $v$ )
2:    $B \leftarrow \text{BuildBlock}(\text{mempool}, \text{proofs})$  ▷ Include ZK proofs
3:   broadcast  $\langle \text{PREPARE}, v, B, \text{highQC} \rangle$ 
4:   wait for  $2f + 1$  PREPARE votes
5:    $\text{prepareQC} \leftarrow \text{CreateQC}(\text{votes})$ 
6:   broadcast  $\langle \text{PRE-COMMIT}, v, \text{prepareQC} \rangle$ 
7:   wait for  $2f + 1$  PRE-COMMIT votes
8:    $\text{precommitQC} \leftarrow \text{CreateQC}(\text{votes})$ 
9:   broadcast  $\langle \text{COMMIT}, v, \text{precommitQC} \rangle$ 
10:  wait for  $2f + 1$  COMMIT votes
11:   $\text{commitQC} \leftarrow \text{CreateQC}(\text{votes})$ 
12:   $\text{FinalizeBlock}(B, \text{commitQC})$ 
13: end procedure
```

4.2 Validator Set Management

Definition 4.2 (Validator Registration). *Any entity can register as a validator by:*

1. *Staking minimum $S_{\min}$ ATLAS tokens*
2. *Publishing validator public key and network endpoint*
3. *Meeting hardware/bandwidth requirements*

No approval or election is required.

The active validator set V_{active} for epoch e consists of the top N validators by total stake:

$$V_{\text{active}}^{(e)} = \text{TopN}(\{(v, s_v + d_v) : v \in V_{\text{registered}}\}, N) \quad (6)$$

where s_v is self-stake and d_v is delegated stake for validator v .

4.3 Leader Selection

Leaders are selected deterministically using stake-weighted random sampling:

$$\text{Leader}(v) = V_{\text{active}}[i] \text{ where } i = \text{VRF}(\text{epoch_seed}, v) \bmod |V_{\text{active}}| \quad (7)$$

weighted by stake proportion to ensure fair opportunity distribution.

4.4 Slashing Conditions

Table 3: Slashing Conditions and Penalties

Violation	Slash Percentage	Jail Duration
Double signing	33%	Permanent
Extended downtime ($> 24\text{h}$)	1%	7 days
Invalid block proposal	50%	Permanent
Censorship (provable)	10%	30 days

5 Privacy-Preserving Transactions

5.1 Private State Model

Private transactions operate on a UTXO-like note model within the commitment tree:

Definition 5.1 (Private Note). A private note $note = (v, asset, \rho, r, pk_{owner})$ contains:

- $v \in \mathbb{F}_p$: Value (amount)
- $asset \in \mathbb{F}_p$: Asset identifier
- $\rho \in \mathbb{F}_p$: Unique note identifier (nullifier seed)
- $r \in \mathbb{F}_p$: Randomness for commitment
- pk_{owner} : Owner's public key

Notes are stored as commitments in a Merkle tree:

$$cm = H(H(pk_{owner}) || v || asset || \rho || r) \quad (8)$$

Definition 5.2 (Nullifier). The nullifier for note under secret key sk is:

$$nf = H(sk || \rho) \quad (9)$$

Publishing a nullifier marks the note as spent without revealing which note.

5.2 Private Transaction Circuit

A private transaction proves the following relation R_{transfer} :

$$R_{\text{transfer}} = \left\{ \begin{array}{l} ((rt, \{nf_i\}, \{cm_j\}, asset), \\ (\{note_i^{\text{in}}\}, \{note_j^{\text{out}}\}, \{path_i\}, sk)) : \\ \forall i : \text{MerkleVerify}(rt, cm_i^{\text{in}}, path_i) = 1 \\ \forall i : nf_i = H(sk || note_i^{\text{in}}. \rho) \\ \forall j : cm_j = \text{Commit}(note_j^{\text{out}}) \\ \sum_i note_i^{\text{in}}.v = \sum_j note_j^{\text{out}}.v \\ \forall i, j : note_i^{\text{in}}.asset = note_j^{\text{out}}.asset = asset \end{array} \right\} \quad (10)$$

5.3 STARK Proof System

We employ the FRI-based STARK protocol [1] for proof generation:

Table 4: STARK Proof System Parameters

Parameter	Value
Field	$\mathbb{F}_p$ where $p = 2^{64} - 2^{32} + 1$ (Goldilocks)
Hash function	BLAKE3
Blowup factor	8
Number of queries	30
Proof size	~50-200 KB
Verification time	~2-5 ms
Security level	100+ bits

5.4 Encrypted Outputs

Recipients learn of incoming notes through encrypted outputs attached to transactions:

$$\text{enc_note} = \text{Enc}_{pk_{\text{recipient}}}(\text{note}) \quad (11)$$

We use Kyber [3] for post-quantum secure key encapsulation.

6 Supply Auditability and Transparency

6.1 Motivation for Supply Auditability

Users, validators, and ecosystem participants must be able to independently verify:

1. Total supply of ATLAS and SHRUG at any point in time
2. Emission rates match the published schedule
3. No unauthorized token creation has occurred
4. Burn mechanisms function correctly

6.2 ATLAS Supply Auditability

Definition 6.1 (ATLAS Supply Formula). *The total ATLAS supply at block height h is deterministically computed as:*

$$S_{ATLAS}(h) = S_{initial} + \sum_{i=0}^h BlockReward(i) + \sum_{i=0}^h StakingRewards(i) - \sum_{i=0}^h Burned(i) \quad (12)$$

where $S_{initial} = 500,000,000$ ATLAS (genesis allocation).

Every block header contains a `supply_commitment` field:

```
1 pub struct BlockHeader {  
2     // ... other fields  
3     pub atlas_supply_root: Hash,           // Merkle root of supply proof  
4     pub atlas_total_supply: u64,          // Claimed total supply  
5     pub atlas_supply_delta: i64,         // Change from previous block  
6 }
```

Listing 1: Block Header Supply Fields

Algorithm 2 ATLAS Supply Verification

```
1: procedure VERIFYATLASSUPPLY(block)  
2:   calculated_delta ← block.coinbase_amount − block.burned_fees  
3:   require calculated_delta = block.header.atlas_supply_delta  
4:   prev_supply ← GetPreviousBlock(block).header.atlas_total_supply  
5:   require block.header.atlas_total_supply = prev_supply + calculated_delta  
6:   expected_reward ← ComputeBlockReward(block.height)  
7:   require block.coinbase_amount ≤ expected_reward + tolerance  
8:   return true  
9: end procedure
```

6.3 SHRUG Supply Auditability

Definition 6.2 (SHRUG Supply Formula).

$$S_{SHRUG}(h) = S_{initial} + \sum_{i=0}^h ProvingRewards(i) + \sum_{i=0}^h DemandBonus(i) - \sum_{i=0}^h Burned(i) \quad (13)$$

where $S_{initial} = 200,000,000$ SHRUG.

The demand multiplier is deterministically computed from on-chain data:

$$\text{multiplier}(\text{epoch}) = \max \left(0.5, \min \left(2.0, 1 + \frac{\text{proofs}_{\text{epoch}} - \text{target}}{\text{target}} \right) \right) \quad (14)$$

6.4 Privacy-Preserving Supply Verification

For SHRUG tokens in the privacy pool, supply verification requires special handling through zero-knowledge supply proofs:

$$R_{\text{supply_audit}} = \left\{ \begin{array}{l} (\text{shielded_balance}, \text{commitment_root}), \\ (\text{all_note_values}, \text{all_randomness}) : \\ \forall \text{notes} : \text{commitment_root contains Commit}(\text{note_value}, \text{randomness}) \\ \wedge \sum(\text{note_values}) = \text{shielded_balance} \\ \wedge \text{All note_values} \geq 0 \end{array} \right\} \quad (15)$$

7 Zero-Knowledge Proof Generation Pipeline

7.1 Overview

This section provides a comprehensive walkthrough of ZK proof generation:

Rust Code $\rightarrow$ Bytecode $\rightarrow$ AIR Constraints $\rightarrow$ Execution Trace $\rightarrow$ FRI Polynomial $\rightarrow$ STARK Proof
(16)

7.2 Step 1: Rust Circuit Definition

```
1 pub struct PrivateTransferCircuit {
2     // Public inputs (visible to verifier)
3     pub merkle_root: [u8; 32],
4     pub nullifiers: Vec<[u8; 32]>,
5     pub new_commitments: Vec<[u8; 32]>,
6     pub asset_id: u64,
7
8     // Private inputs (witness, not revealed)
9     notes_in: Vec<Note>,
10    notes_out: Vec<Note>,
11    merkle_paths: Vec<MerklePath>,
12    secret_key: [u8; 32],
13 }
14
15 impl PrivateTransferCircuit {
16     pub fn build_trace(&self) -> TraceTable {
17         let mut trace = TraceTable::new(TRACE_WIDTH, TRACE_LENGTH);
18
19         // Step 1: Verify Merkle paths for input notes
20         for (i, (note, path)) in self.notes_in.iter()
21             .zip(self.merkle_paths.iter()).enumerate() {
22             let commitment = self.compute_commitment(note);
23             let computed_root = self.verify_merkle_path(commitment, path);
24             trace.set(ROW_START + i * STEPS_PER_NOTE, COL_ROOT, computed_root);
25         }
26
27         // Step 2: Verify value conservation
28         let sum_in: u64 = self.notes_in.iter().map(|n| n.value).sum();
29         let sum_out: u64 = self.notes_out.iter().map(|n| n.value).sum();
30         trace.set(ROW_BALANCE, COL_SUM_IN, sum_in);
31         trace.set(ROW_BALANCE, COL_SUM_OUT, sum_out);
32
33         trace
34     }
35 }
```

7.3 Step 2: Algebraic Intermediate Representation (AIR)

Definition 7.1 (AIR Constraints). *An AIR defines polynomial constraints C_i that must equal zero for every row of the execution trace:*

$$\forall t : C_i(\text{trace}[t], \text{trace}[t+1], \dots, \text{trace}[t+k]) = 0 \quad (17)$$

7.4 SVM Instruction ZK-Compatibility

Theorem 7.2 (SVM Instruction ZK-Compatibility). *Every SVM instruction can be represented as an AIR constraint system, enabling zero-knowledge execution of arbitrary SVM programs.*

Proof Sketch. 1. **Deterministic Execution:** SVM instructions are deterministic functions

2. **Algebraic Representation:** Each instruction decomposes into field operations

3. **Instruction-to-AIR Mapping:** Every instruction maps to polynomial constraints $\square$

7.5 Performance Characteristics

Table 5: ZK Proving Performance by Operation Type

Operation Type	Complexity	Proof Overhead	Example
Field arithmetic (+, -, ×)	Fast (1-10 cycles)	Low	Balance calculations
Hash functions (Poseidon)	Medium (100-500)	Medium	Commitment generation
Hash functions (SHA-256)	Very Slow (10K+)	Very High	Legacy hash verification
Signature verification	Slow (5K+ cycles)	High	EdDSA verification
Range checks	Medium (50-200)	Medium	Value bounds checking
Division/Modulo	Slow (500+ cycles)	High	Requires extended GCD
Bitwise operations	Slow (10-50/bit)	High	Binary decomposition

7.6 End-to-End Proving Timeline

For a typical private transfer (2 inputs, 2 outputs):

Table 6: Proving Timeline on RTX 4090

Operation	Time	GPU Utilization
Circuit setup (one-time)	50 ms	5%
Witness generation	10 ms	10%
Trace generation	200 ms	40%
Trace commitment (Merkle tree)	100 ms	60%
Constraint polynomial computation	500 ms	80%
FRI commitment layers	1,500 ms	95%
Query proof generation	300 ms	70%
Proof serialization	40 ms	5%
Total Proving Time	2.7 seconds	
Proof Size	~120 KB	
Verification (validators)	3 ms	

8 Network Architecture

8.1 Maximum Block Size and Packet Specifications

Definition 8.1 (Block Size Limits).

$$MAX_BLOCK_SIZE = 2 \text{ MB} = 2,097,152 \text{ bytes} \quad (18)$$

$$MAX_PUBLIC_TX_PER_BLOCK = 5,000 \quad (19)$$

$$MAX_PRIVATE_TX_PER_BLOCK = 100 \quad (20)$$

$$TARGET_BLOCK_TIME = 2 \text{ seconds} \quad (21)$$

8.2 Gossip Protocol Packet Structure

```

1 pub struct GossipPacket {
2     pub magic: [u8; 4],           // Protocol identifier: "RAND"
3     pub version: u8,              // Protocol version
4     pub message_type: MessageType, // Tx, Vote, Proof, Block, etc.
5     pub compression: CompressionType, // None, LZ4, Zstd
6     pub payload_size: u32,        // Uncompressed size
7     pub payload: Vec<u8>,         // Actual data
8     pub checksum: [u8; 32],       // BLAKE3 hash
9 }
10
11 pub const MIN_PACKET_SIZE: usize = 64;           // Minimum valid packet
12 pub const MAX_PACKET_SIZE: usize = 262_144;     // 256 KB unchunked
13 pub const MAX_CHUNKED_MSG: usize = 2_097_152;  // 2 MB for full blocks

```

Listing 3: Gossip Packet Structure

8.3 Performance Metrics

Table 7: System Performance Targets

Component	Latency	Throughput
<i>Public Transaction</i>		
Signature verification	0.1 ms	10,000/sec per core
Mempool propagation	50-200 ms	Network bound
SVM execution	0.04 ms	25,000/sec per core
Block inclusion	2 seconds	5,000/block
<i>Private Transaction</i>		
ZK proof generation	3-5 seconds	20,000 proofs/day/GPU
Proof verification	2-5 ms	200-500/sec per core
Block inclusion	2 seconds	100/block
<i>Consensus</i>		
BFT round trip	~500 ms	1 round
Total finality	2-6 seconds	3 rounds
Block propagation	50-200 ms	2 MB/block

8.4 Bandwidth Requirements

- **Validator Node:** 1 MB/s sustained (8 Mbps), ~86 GB/day
- **GPU Prover:** 500 KB/s upload, ~10 GB/day
- **Light Client:** 10 KB/s (block headers only), ~860 MB/day

9 Token Economics

9.1 Initial Token Distribution

Table 8: Initial Token Distribution

Allocation	ATLAS (%)	SHRUG (%)
Core Development Team	15	15
Foundation/Treasury	20	20
Ecosystem Development	15	15
Early Validators/Provers	5	10
Private Sale	10	5
Public Sale	10	10
Community Airdrops	5	5
Staking Rewards Reserve	20	20
Total Initial Supply	500M ATLAS	200M SHRUG

9.2 Inflation Model

Definition 9.1 (Inflation Schedule). *The annual inflation rate $\iota(t)$ at year t follows:*

$$\iota(t) = \max\{\iota_\infty, \iota_0 \cdot (1 - \delta)^t\} \quad (22)$$

where ι_0 is initial inflation, δ is annual reduction rate, and ι_∞ is long-term inflation.

ATLAS Inflation Parameters:

- Initial inflation (ι_0): 8% annually
- Reduction rate (δ): 15% per year
- Long-term inflation (ι_∞): 1.5% annually
- Target staking rate: 65% of circulating supply

9.3 SHRUG Demand-Driven Emission

$$\text{Emission}_{\text{SHRUG}}(e) = \text{Base_Emission}(e) \cdot \min\left(2, 1 + \frac{\text{Proofs}(e) - \text{Target_Proofs}}{\text{Target_Proofs}}\right) \quad (23)$$

9.4 Fee Distribution

Table 9: Fee Distribution by Transaction Type

Fee Type	Proposer	Voters	Treasury	Burn
Public tx gas (ATLAS)	60%	20%	10%	10%
Private tx base (SHRUG)	40%	–	30%	10%
Proving fee (SHRUG)	80%*	–	20%	–

*Goes to prover, not proposer

10 Security Analysis

10.1 Consensus Security

Theorem 10.1 (Safety). *If fewer than $f < n/3$ validators are Byzantine, no two honest validators commit to conflicting blocks.*

Proof. The HotStuff protocol requires $2f + 1$ signatures for each QC. For two conflicting blocks B and B' to both receive commitQCs:

$$|S_B| \geq 2f + 1 \quad (24)$$

$$|S_{B'}| \geq 2f + 1 \quad (25)$$

Since honest validators sign at most one block per view:

$$|S_B \cap S_{B'}| \geq (2f + 1) + (2f + 1) - n = 4f + 2 - (3f + 1) = f + 1 \quad (26)$$

This implies at least $f + 1$ validators signed both, contradicting the assumption that at most f are Byzantine. $\square$

Theorem 10.2 (Liveness). *Under partial synchrony, if fewer than f validators are Byzantine and the network eventually becomes synchronous, the protocol makes progress.*

10.2 Privacy Security

Theorem 10.3 (Transaction Privacy). *Under the security of the hash function H and the STARK proof system, private transactions reveal only the public inputs (nullifiers, commitments, Merkle root) and nothing about the sender, recipient, or amount.*

Theorem 10.4 (Double-Spend Prevention). *A valid transaction cannot spend the same note twice.*

Proof. Each note has a unique nullifier $\text{nf} = H(\text{sk} \parallel \rho)$. The circuit enforces correct nullifier computation. Validators maintain a nullifier set $\mathcal{N}$ and reject transactions with $\text{nf} \in \mathcal{N}$. After inclusion, nf is added to $\mathcal{N}$. Any re-spend produces the same nullifier, which is rejected. $\square$

10.3 Post-Quantum Security

Proposition 10.5. *The system maintains security against quantum adversaries using:*

- *Signatures: CRYSTALS-Dilithium (lattice-based)*
- *Key encapsulation: CRYSTALS-Kyber (lattice-based)*
- *ZK proofs: STARKs (hash-based)*
- *Hash functions: SHA-3, BLAKE3 (128-bit quantum security via Grover bound)*

11 Implementation Considerations

11.1 Development in Rust

All core components are implemented in Rust, leveraging:

- Memory safety without garbage collection
- Zero-cost abstractions
- Fearless concurrency
- Strong type system

11.2 Performance Targets

Table 10: System Performance Targets

Metric	Target
Block time	2 seconds
Finality time	2-6 seconds
Public TPS	5,000-10,000
Private TPS	100-500 (proof-limited)
Active validators	50-100
Proof verification	<5 ms

12 Related Work

Consensus Mechanisms. Bitcoin [14] introduced Nakamoto consensus with probabilistic finality. Ethereum 2.0 combines PoS with Casper FFG for finality. Tendermint [4] and HotStuff [19] provide BFT consensus with deterministic finality. Our work combines these approaches with useful work.

Privacy Cryptocurrencies. Zcash [9] pioneered zk-SNARK-based privacy. Monero [15] uses ring signatures and stealth addresses. Aztec Protocol [17] brings privacy to Ethereum. Our approach differs in using STARKs for post-quantum security.

Proof-of-Useful-Work. Prior PoUW proposals include Primecoin [11] and AI-PoW systems [13]. Our approach uniquely ties useful work to the system’s core functionality (privacy proofs).

13 Conclusion

We have presented Rand, a blockchain architecture that achieves the seemingly contradictory goals of fast finality and permissionless participation through a novel hybrid consensus mechanism. By transforming ZK proof generation into useful work, we create a permissionless entry point (GPU proving) while maintaining BFT consensus efficiency.

The dual-token model aligns incentives: validators secure consensus and earn ATLAS, while GPU provers enable privacy and earn SHRUG. The integration with SVM provides smart contract functionality and ecosystem compatibility.

Future work includes formal verification of the implementation, optimization of the STARK proving system for specific GPU architectures, and exploration of recursive proof composition for enhanced scalability.

Acknowledgments

We thank the broader cryptocurrency research community for foundational work on consensus mechanisms, zero-knowledge proofs, and privacy-preserving systems.

References

- [1] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, “Scalable, transparent, and post-quantum secure computational integrity,” in *CRYPTO*, 2018.
- [2] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, “Keccak,” in *EUROCRYPT*, 2013.

- [3] J. Bos et al., “CRYSTALS-Kyber: A CCA-secure module-lattice-based KEM,” in *Euro S&P*, 2018.
- [4] E. Buchman, “Tendermint: Byzantine fault tolerance in the age of blockchains,” PhD thesis, 2016.
- [5] V. Buterin, “On sharding blockchains,” Ethereum Foundation, 2017.
- [6] L. Ducas et al., “CRYSTALS-Dilithium: A lattice-based digital signature scheme,” *TCHES*, 2018.
- [7] Y. Gilad et al., “Algorand: Scaling byzantine agreements for cryptocurrencies,” in *SOSP*, 2017.
- [8] L. K. Grover, “A fast quantum mechanical algorithm for database search,” in *STOC*, 1996.
- [9] D. Hopwood et al., “Zcash protocol specification,” 2016.
- [10] A. Kiayias et al., “Ouroboros: A provably secure proof-of-stake blockchain protocol,” in *CRYPTO*, 2017.
- [11] S. King, “Primecoin: Cryptocurrency with prime number proof-of-work,” 2013.
- [12] L. Lamport, R. Shostak, and M. Pease, “The Byzantine generals problem,” *TOPLAS*, 1982.
- [13] A. Lihu, J. Du, and S. Barber, “A proof of useful work for AI on the blockchain,” 2020.
- [14] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008.
- [15] S. Noether, “Ring signature confidential transactions for Monero,” *IACR ePrint*, 2015.
- [16] J. O’Connor et al., “BLAKE3: One function, fast everywhere,” 2020.
- [17] Z. J. Williamson, “The Aztec protocol,” 2018.
- [18] A. Yakovenko, “Solana: A new architecture for a high performance blockchain,” 2018.
- [19] M. Yin et al., “HotStuff: BFT consensus with linearity and responsiveness,” in *PODC*, 2019.